

最后一个输入，还要算一次？

先认清：最后一个输入 token，与第一个输出 token，是两个位置。

这句话究竟是什么意思

KV 缓存保留了可复用的中间资料。要决定接下来输出什么，模型还需要最后一个输入位置的最终计算结果。

假设用户输入：今天天气很



为了示意，假设每个方块是 1 个 token；真实分词未必这样切。

重新算的是用户已经给出的“很”。模型根据“今天 + 天气 + 很”，为后面可能出现的 token 打分，再按生成规则选出“好”。

候选输出	示意评分：logits	如何理解
好	3.1	当前较倾向这个候选
冷	2.0	也是一个候选
桌子	-1.4	当前较不倾向这个候选

logits 就是一串原始分数，不是百分比。上表是教学数字；采样规则也会影响最后选哪个 token。

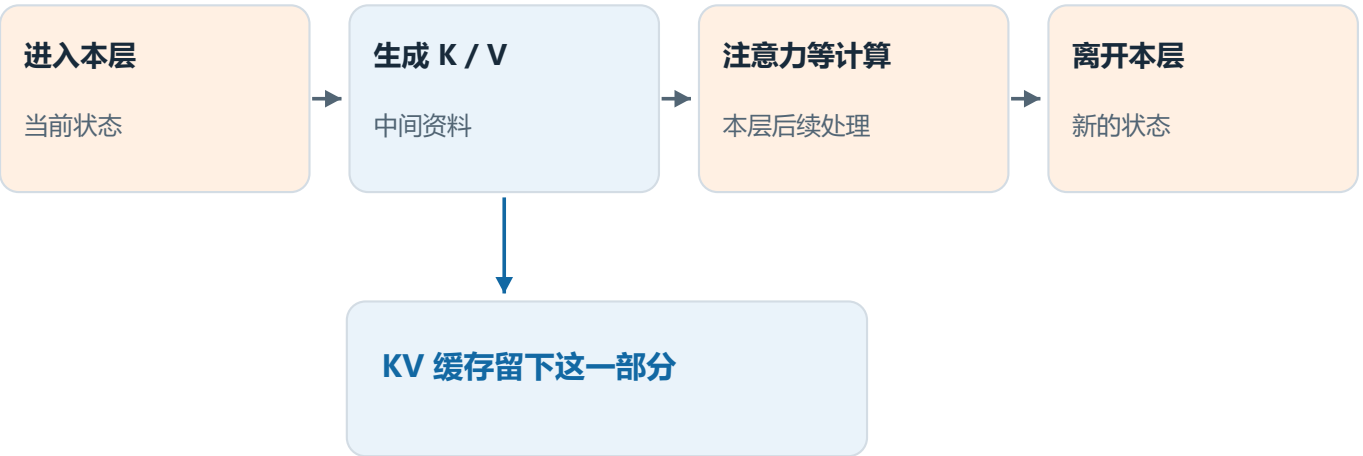
选出“好”后，如果继续生成，下一轮才会把“好”作为新的输入位置，预测它后面的 token。

KV 是资料，logits 是候选评分

可以把 KV 理解成各层留下的“可查阅笔记”；这只是帮助理解的比喻。

名称	在模型里做什么	普通 KV 缓存保存吗？
K / V	K 帮当前 token 找相关信息；V 提供被取出的信息。 实际都是数字向量。	保存每层的 K 和 V
最终隐藏状态	最后一个输入位置走完所有模型层后形成的结果。	不属于普通 KV 缓存
logits	将最终隐藏状态交给输出层，得到整个词表的候选评分。	不属于普通 KV 缓存

一层计算里，KV 只是中间的一部分



流程为概念示意；省略残差、归一化等细节。KV 分支不会保存完整的层输出。

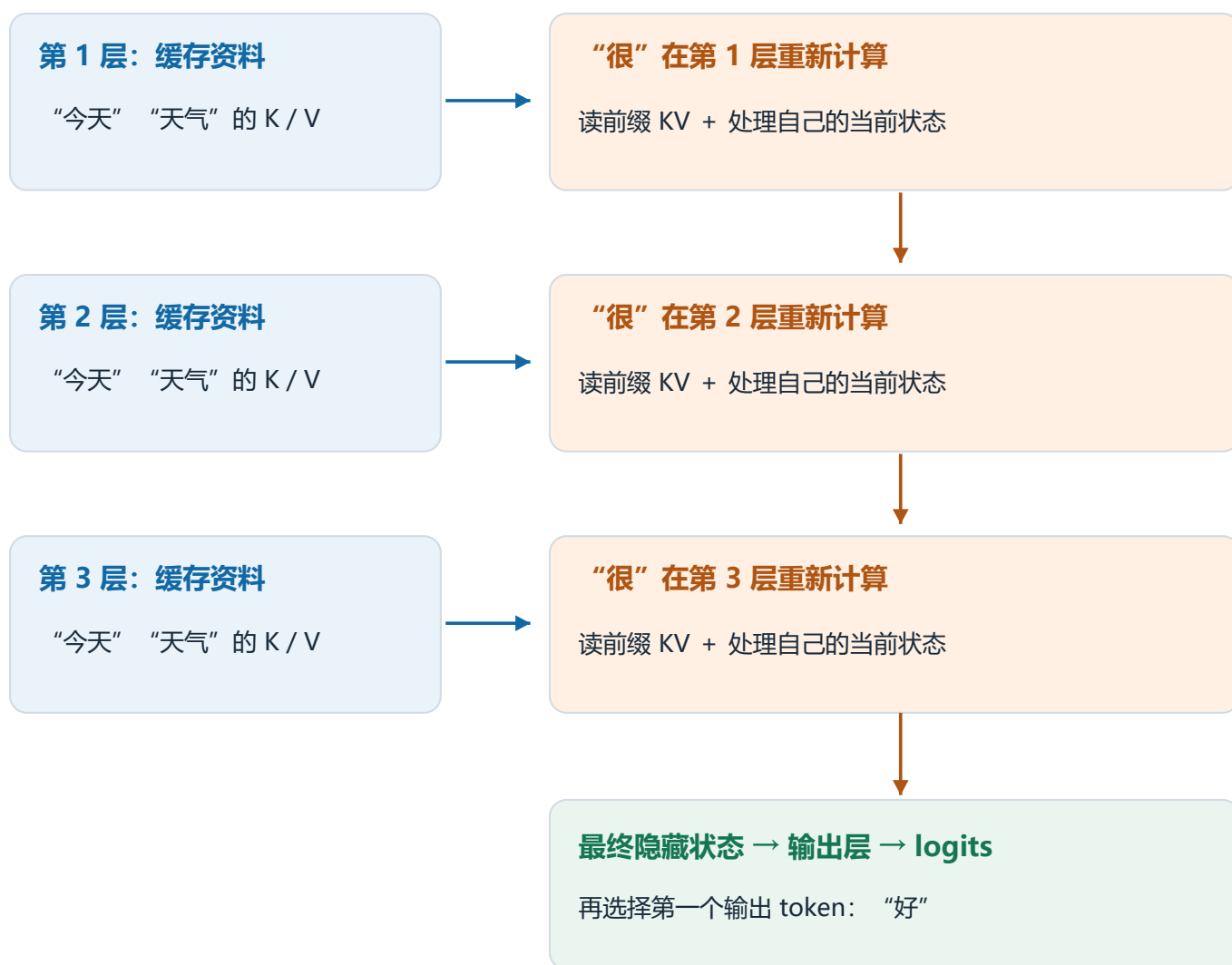
所以：缓存里有最后位置的 KV，也不等于有最终评分

下一次请求能够复用前缀的 KV，但普通 KV 缓存没有直接给出这次所需的最终隐藏状态和 logits。vLLM 因此仍安排尾部 token 的前向计算。

重算一个 token，也要走完整个模型

先看理想情况：只复用前 N-1 个 token，最后一个位置单独重新计算。

仍用“今天 | 天气 | 很”。假设前两项在每一层的 KV 都可复用。下面把模型缩成 3 层示意；实际层数由模型决定。



前缀省下的是重新生成它们的 KV；本次尾部位置仍需在每层读取相关 KV。它还会生成自己的本层 KV，供当前注意力和后续生成使用。

不是只跑最后一层，也不是先把“好”当输入来重算。

vLLM 实际可能重算一段尾巴

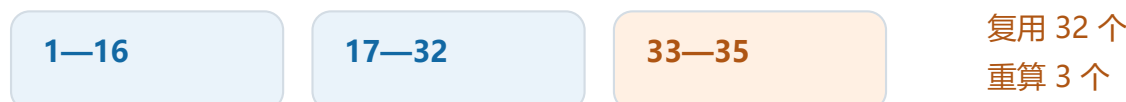
示例限定：普通全注意力、本地前缀缓存，有效命中对齐粒度为 16 token。

当前核对的 vLLM 版本会先限制：最多复用到最后一个输入 token 之前。然后按该缓存路径的对齐规则，找可复用前缀。

$$\text{max_cache_hit_length} = \text{request.num_tokens} - 1$$

下面假设所需前缀缓存都存在。蓝色 = 复用，橙色 = 本次重新计算。图表示位置范围，不按块内 token 数绘制长度。

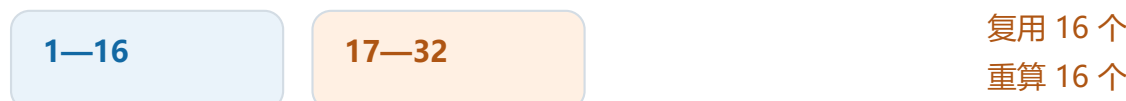
输入 35 个 token



复用 32 个
重算 3 个

最多命中 34 个；向下对齐到 16 的倍数，得到 32。

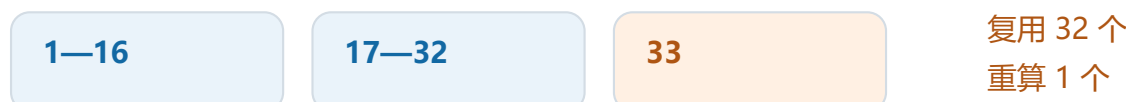
输入 32 个 token



复用 16 个
重算 16 个

最多命中 31 个；向下对齐到 16 的倍数，得到 16。

输入 33 个 token



复用 32 个
重算 1 个

最多命中 32 个；向下对齐到 16 的倍数，得到 32。

因此：至少重算最后一个位置，不等于每次只重算一个 token。整块命中时，最后一块也可能被留给本次重算。

16 是教学配置，不是所有设备的默认值。细粒度命中、混合模型、推测解码或外部 KV 连接器可能使用其他规则；不要把这个示例当作所有配置的统一公式。

回到我们的 ToolAgent 文件

文件里的 miss=1 是转换模型的安排，不能直接当成 vLLM 运行测量。

23,608 条请求中，256 条被转换为只计算 1 个 token。转换器按历史 hash 判断前缀已见过，并把命中上限设成“总长度减 1”；这不是原 trace 记录的实测计算量。

同样的输入长度	当前转换的计算量	第 4 页的假设：对齐 16
6,257 token	1 token	命中 6,256；重算 1
78,964 token	1 token	命中 78,960；重算 4

右列是依据对齐规则推算，不是已经运行 vLLM 的结果；前提仍是相关前缀缓存均可复用。

只重算 1 个，为什么仍可能需要读很多？

request_id=16797：输入 6,257 token，其中 6,256 个命中。当前合成模型给它每层约 53.83 微秒计算、24.44 MiB KV。若要把读取藏进这段计算，需要约 443.37 GiB/s。

KV 已在本卡 HBM

需要本地读取；不必再从 SSU 取同一份。

KV 只在 SSU / 远端

还要搬到执行侧；可能产生外部 I/O 等待。

这 53.83 微秒来自合成公式，不是 vLLM 实测。命中层级、有效块粒度、真实算子时间不同，计算量和外部带宽诉求都会变。

现在可以准确理解原来那句话了

普通 KV 缓存复用前缀资料，尾部计算负责把资料变成当前需要的输出评分。

完整的一句话

即使输入对应的 KV 都曾经算过，普通 KV 缓存也没有直接保存所需的最终 logits。vLLM 会保留尾部 token 的计算，以取得最后一个输入位置的输出；实际重算多少，还取决于命中对齐规则。

常见疑问	准确理解
重算的是哪个 token？	至少包括最后一个输入 token “很”；按块对齐时还包括前面的尾段。不是回答 “好”。
为什么不能直接拿最后一个 V 当答案？	V 是一层注意力使用的中间向量，不是词表评分；后续计算和最终输出层仍要执行。
既然以前算过 logits，为什么不用？	普通 KV 缓存没有缓存它。另存最终隐藏状态或 logits 是另一种缓存设计，并非理论上绝对禁止。
重算 16 个 token，时间就是 16 倍吗？	不是。多个新 token 通常可以并行处理；时间取决于模型、上下文、硬件和 batch。

核对一下：输入有 32 个 token，有效对齐粒度为 16，且前缀缓存齐全。按第 4 页规则，会复用 16 个、重算 16 个。你能解释为什么不是全部复用吗？

依据与适用范围

vLLM：为什么把命中上限设为 $N-1$ (点击)

vLLM：缓存协调与 full-attention 对齐 (点击)

vLLM：模型层与 hidden states $\rightarrow$ logits (点击)

官方文档：自动前缀缓存 (点击)

源码核对：2026-09-19；提交 a8d1aa9c99b8。本文没有运行真实 vLLM 性能测试，示意 token 切分和候选分数也不是模型实测。